



ISLAMIA UNIVERSITY OF BAHAWALPUR

Title:

Hotel Management and Smart Room Booking System

Submitted By:

Moazam Ajmal

: Submitted To:

DR. Abdul Kareem Nawaz

Roll No:

S25BINFT7M04032

Semester:

7th(1 M) ADP

Table of Contents

1. Introduction	iv
1.1 Purpose	iv
1.2 Scope	iv
1.3 Definitions, Acronyms, and Abbreviations	v
1.4 References	v
1.5 Overview	v
2. The Overall Description	v
2.1 Product Perspective	v
2.1.1 Operations	vi
Guest Mode (Interactive):	vi
Admin Mode (Interactive):	vi
AI Processing (Backend, Unattended):	vi
2.1.2 Site Adaptation Requirements	vi
2.2 Product Functions	vi
2.3 User Characteristics	vii
2.4 General Constraints	vii
2.5 Assumptions and Dependencies	vii
3. Specific Requirements	viii
3.1 External Interface Requirements	viii
3.1.1 System Interfaces	viii
3.1.2 Interfaces	viii
3.1.3 Hardware Interfaces	viii
3.1.4 Software Interfaces	viii
3.1.5 Communications Interfaces	ix
3.3 Use Cases	xii
3.3.1 Use Case	xii
1: User Registration & Login	xii
3.3.2 Use Case	xiii
2: Book Room	xiii
3.3.3 Use Case	xiii
3: Manage Inventory	xiii

		xiv
3.3.4 Use Case	xiv	
4: Notification Service		xiv
3.3.5 Use Case	xv	
5: AI Personalized Room Sorting		xv
3.3.6 Use Case	xv	
6: Cash on Counter Payment		xv
7: Online Payment		xv
3.4 Classes / Objects		xvi
3.4.1 Class #1: User	xvi	
3.4.2 Class #2: Room	xvi	
3.5 Non-Functional Requirements		xvii
3.5.1 Performance	xvii	
3.5.2 Reliability	xvii	
3.5.3 Availability	xvii	
3.5.4 Security	xvii	
3.5.5 Maintainability	xvii	
3.5.6 Portability	xvii	
3.6 Inverse Requirements	xvii	
3.7 Logical Database Requirements	xviii	
3.8 Design Constraints	xviii	
3.8.1 Standards Compliance		xviii
Change Management Process		xix
A. Appendices	xxi	
A.1 Glossary	xxi	
A.2 Technology Stack Details	xxii	
A.3 Sample Data Structures	xxii	

1. Introduction

1.1 Purpose

The purpose of this SRS is to define the requirements for the "Hotel Management and Smart Room Booking System". This document is intended for the project development team (student) and the project supervisor (Dr. Abdul Karim Nawaz) to ensure a clear understanding of the system's features, specifically the AI-based recommendation engine.

1.2 Scope

The software product to be produced is a web-based application named "SmartBook".

Product Functionality:

Allow guests to search for and book rooms.

Enable administrators to manage hotel inventory (rooms, pricing, availability).

Integrate an AI module to personalize room sorting and provide context-aware upselling suggestions.

Objectives:

Reduce decision fatigue for users by showing relevant rooms first.

Maximize hotel revenue using targeted, context-aware upgrade suggestions.

1.3 Definitions, Acronyms, and Abbreviations

SRS: Software Requirements Specification.

AI Helper: The machine learning component that analyzes user data.

Upselling: Encouraging a customer to purchase a higher-end product or add-on.

Admin: Hotel staff responsible for managing the system.

1.4 References

IEEE Guide to Software Requirements Specification (ANSI/IEEE Std. 830-1984).

SmartBook Project Proposal (Moazam Ajmal, Session 2025-27).

Scikit-learn Documentation (for AI model references).

1.5 Overview

This document describes the external interfaces, functional requirements, and non-functional requirements of the system. Section 2 provides a general context, while Section 3 details specific requirements for developers and testers.

2. The Overall Description

2.1 Product Perspective

SmartBook is a **self-contained web-based hotel management and smart room booking system**. The system operates in a **client-server architecture**, where users interact with the application through a **web browser**, and all processing is handled on the server side.

The presentation layer is implemented using **HTML5, CSS3, and JavaScript**, providing an interactive and responsive user interface. The application layer is implemented using **PHP**, which handles business logic, authentication, booking workflows, and AI-related processing. The system connects to a centralized **relational database (MySQL/PostgreSQL)** for persistent data storage.

The **AI recommendation functionality** is integrated into the backend and operates as a background service. The AI components are hidden from end-users and function as a **black box**, providing personalized room recommendations and upsell suggestions without exposing internal logic.

2.1.1 Operations

The system supports the following operational modes:

Guest Mode (Interactive):

- Users can search for available rooms based on date and preferences.
- Users can view room details and AI-based room recommendations.
- Users can book rooms and select a payment method.
- Users can accept or decline AI-generated upsell suggestions.

Admin Mode (Interactive):

- Hotel staff can add, update, and manage room inventory.
- Administrators can set pricing, update availability, and manage bookings.
- Administrators can view booking records and basic statistics.
- Administrators can manage check-in and check-out operations.

AI Processing (Backend, Unattended):

- AI recommendation logic runs in the backend as part of the application layer.
- User behavior, booking history, and room data are analyzed to generate recommendations.
- AI processing is triggered during room search and booking operations.

2.1.2 Site Adaptation Requirements

Server Setup:

A web server environment supporting **PHP** (Apache/Nginx) is required to host the application and execute backend logic.

Database:

A relational database system such as **MySQL or PostgreSQL** is required for data storage.

Database Initialization:

Initial SQL scripts must be executed to create required tables, including **Users, Rooms, Bookings, Payments, Transactions, and Logs**.

2.2 Product Functions

Major system functions include:

User Management: User registration, authentication (login/logout), profile management, and role-based access (Guest/Admin).

Booking Engine: Room search, filtering by price and amenities, viewing room details, room reservation, and booking confirmation.

AI Recommendation System: Provides personalized room recommendations and intelligent upsell suggestions based on user behavior and booking context.

Payment Handling: Supports online payment simulation and cash-on-counter payment options.

Admin Dashboard: Allows administrators to manage room inventory, pricing, availability, and booking records.

2.3 User Characteristics

Guest:

General public users with basic computer literacy. Users expect a simple, responsive, and mobile-friendly web interface.

Administrator:

Hotel staff with moderate technical skills. Administrators require easy-to-use forms and dashboards for managing hotel operations with minimal errors.

2.4 General Constraints

Performance:

The system should respond quickly to user actions. AI-based recommendation processing should not cause noticeable delay in the user interface.

Concurrency:

The system must prevent double bookings through proper database constraints and transaction handling.

Auditability:

All booking, payment, and administrative actions must be logged for traceability and system monitoring.

2.5 Assumptions and Dependencies

Users access the system using modern web browsers (Chrome, Edge, etc.).

Stable internet connectivity is assumed for normal operation.

Availability of the database server is required for system functionality.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 System Interfaces

The backend system is implemented using **PHP**, which handles core application logic and communicates with the database using secure database access methods.

AI recommendation logic is integrated into the backend and processes data retrieved from the database.

3.1.2 Interfaces

Web GUI:

A responsive web interface developed using **HTML5, CSS3, and JavaScript**. JavaScript is used for form validation and asynchronous interactions (AJAX).

Accessibility:

The interface follows standard accessibility guidelines, including proper contrast ratios and alternative text for visual elements.

3.1.3 Hardware Interfaces

Server:

A standard server or cloud-based virtual machine capable of running a PHP-enabled web server and relational database.

Client:

Desktop and mobile devices with modern web browsers. No specialized hardware is required.

3.1.4 Software Interfaces

Database:

MySQL or PostgreSQL using standard TCP/IP communication ports.

Web Server:

Apache or Nginx with PHP support.

Backend Language:

PHP

3.1.5 Communications Interfaces

Protocol:

HTTP / HTTPS for secure client–server communication.

Data Format:

JSON format for asynchronous request and response handling (AJAX/REST-style communication).

3.2 Functional Requirements

3.2.1 Functional Requirement

1: User Registration & Login

Introduction:

Allow guests and admins to securely register, login, and manage profiles.

Inputs:

Email, password, name, phone

Processing:

Validate credentials

Apply password hashing (bcrypt)

Manage user sessions

Outputs:

Successful login or “Invalid Credentials” message

Error Handling:

Lock account after repeated failures

Show password policy warnings

3.2.2 Functional Requirement

2: AI Personalized Room Sorting

Introduction:

Reorder available room listings based on personalized relevance for each user.

Inputs:

User ID

Search parameters (dates, guests)

List of available rooms

Processing:

Analyze user profile and booking history

Evaluate booking context (dates, duration, device type)

Score each room based on attributes (amenities, bed type, occupancy)

Generate relevance ranking using AI model

Outputs:

A personalized, re-ordered list of room results shown to the user

Error

Handling:

If AI model fails or times out, switch to default sorting (e.g., Price: Low to High)

3.2.3 Functional Requirement

3: Room Availability Checking

Introduction:

The system must verify real-time room availability before completing a booking.

Inputs:

Check-in & check-out dates

Selected room type

Processing:

Query database for room status

Lock the selected room temporarily during booking

Outputs:

Confirmation that room is available OR error message if room is already booked

Error Handling:

If booking conflict occurs, notify user and refresh inventory

3.2.4 Functional Requirement

4: Notification Service

Introduction:

Automatically send email notifications.

Inputs:

Booking status (confirmed, cancelled)

User email address

Processing:

Trigger template-based email service

Outputs:

Confirmation email

Cancellation email

Error Handling:

Log failure and retry sending

3.2.5 Functional Requirement

5: Search & Filter Engine

Introduction:

Users can refine room search results.

Inputs:

City, date range, price range, amenities

Processing:

Query rooms based on filters

Apply AI sorting if enabled

Outputs: Updated room results

Error Handling:

If no matching results → show suggestions

3.2.6 Functional Requirement

6: Payment Processing

Introduction:

System must support both online and cash-on-counter payments.

Inputs:

Payment method, booking details **Processing:**

Tokenize card details (if online)

Validate payment status

Outputs:

Payment success, failure, or pending

Error Handling:

If online payment fails → automatically retry OR allow user to choose another method

3.3 Use Cases

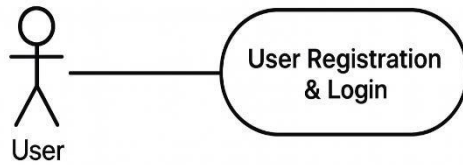
3.3.1 Use Case

1: User Registration & Login

Actor: **User (Guest/Admin)**

Action: User opens the system, enters email and password, or fills out the registration form with required details (email, password, name, phone). The system validates inputs, checks password strength, verifies credentials, applies password hashing, and creates a secure session after successful login.

Outcome: User account is created (for registration), or user is successfully logged in and redirected to the dashboard. If credentials are invalid, the system displays an error message. Account is locked temporarily after multiple failed attempts.

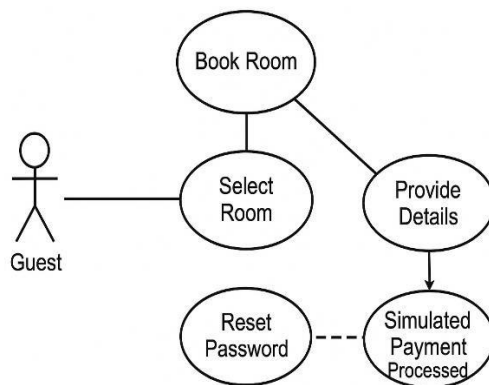


3.3.2 Use Case

2: Book Room

Actor: Guest

Action: Guest searches for a room, receives AI-sorted results, selects a room, sees optional upsell, provides details (guest info), confirms booking, simulated payment processed. **Outcome:** Booking saved (Confirmed), confirmation email/notification sent



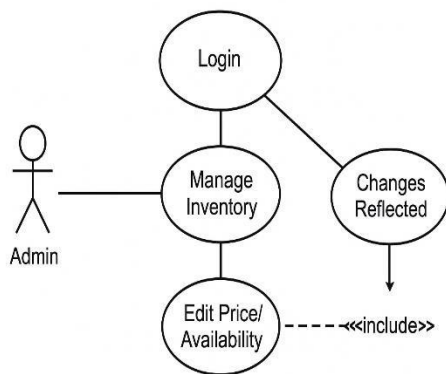
3.3.3 Use Case

3: Manage Inventory

Actor: Admin

Action: Admin logs in, edits room price/availability, sets seasonal pricing or promotions, saves changes.

Outcome: Changes reflected immediately to guest-facing search results.



3.3.4 Use Case

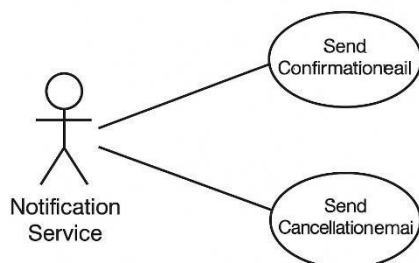
4: Notification Service

Actors: Notification Service Action:

Send Confirmation Email

Send Cancellation Email

Notification Service Use Case Diagram



3.3.5 Use Case

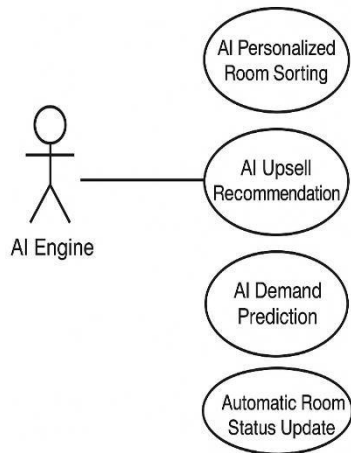
5: AI Personalized Room Sorting

Actor: AI Engine

Action: AI Engine analyzes customer preferences, booking behavior, and room attributes to automatically sort rooms in the most relevant order for the user.

Outcome: Users see a dynamically sorted room list optimized for their preferences.

System / AI Engine Use Case Diagram



3.3.6 Use Case

6: Cash on Counter Payment

Actor: User

Action: User selects the *Cash on Counter* option during checkout, confirms the booking, and completes payment physically at the reception or counter.

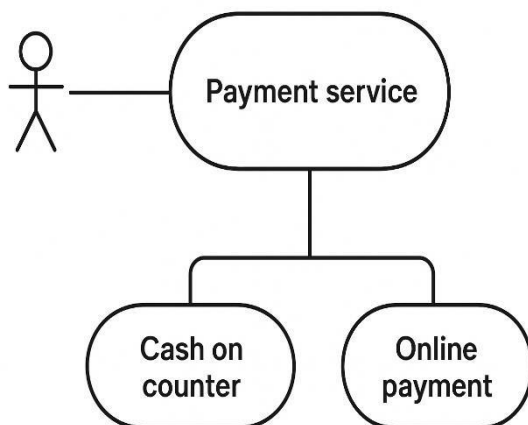
Outcome: Booking is reserved, and payment is marked as *Pending* until confirmed by staff.

7: Online Payment

Actor: User

Action: User selects *Online Payment*, enters card or digital wallet details, completes the online transaction, and receives immediate payment confirmation.

Outcome: Payment is processed instantly, and the booking is confirmed automatically.



3.4 Classes / Objects

3.4.1 Class #1: User

Attributes:

UserID (PK)

Name

Email

PasswordHash

Preferences (List)

History (List of bookings & interactions)

Functions:

Register()

Login()

UpdateProfile()

GetRecommendations() (invokes AI Helper)

3.4.2 Class #2: Room

Attributes:

RoomID (PK)

Type (Standard/Deluxe/Suite)

Price

Amenities (List)

Status (Available/Booked/Maintenance)

OccupancyLimit Functions:

GetDetails()

UpdateStatus()

CalculatePrice() (applies season/promo rules)

3.5 Non-Functional Requirements

3.5.1 Performance

Search results must load in < 3 seconds under normal load.

AI recommendation inference must respond in < 200 ms.

3.5.2 Reliability

Maintain data integrity during concurrent booking operations. Use DB transactions and optimistic/pessimistic locking where needed.

3.5.3 Availability

System availability target: 99% during business hours.

3.5.4 Security

Passwords stored hashed with bcrypt.

Protect against SQL injection, XSS, CSRF.

TLS (HTTPS) for all network traffic.

3.5.5 Maintainability

Backend to follow PEP-8 conventions.

Modular codebase and documented APIs.

3.5.6 Portability

Web application must be accessible on Chrome, and mobile browsers.

3.6 Inverse Requirements

The system shall not allow a booking for dates where no availability exists.

The system shall not store raw credit card numbers; payment is simulated and sensitive data is tokenized or stored off-site if needed.

3.7 Logical Database Requirements

Relational schema with FK constraints.

Primary Keys: UserID, RoomID, BookingID.

Relationships: A User has many Bookings; a Booking references one Room.

3.8 Design Constraints

3.8.1 Standards Compliance

Follow IEEE 830 SRS guidance for structure and presentation.

Use RESTful API conventions for web services.

Analysis Models

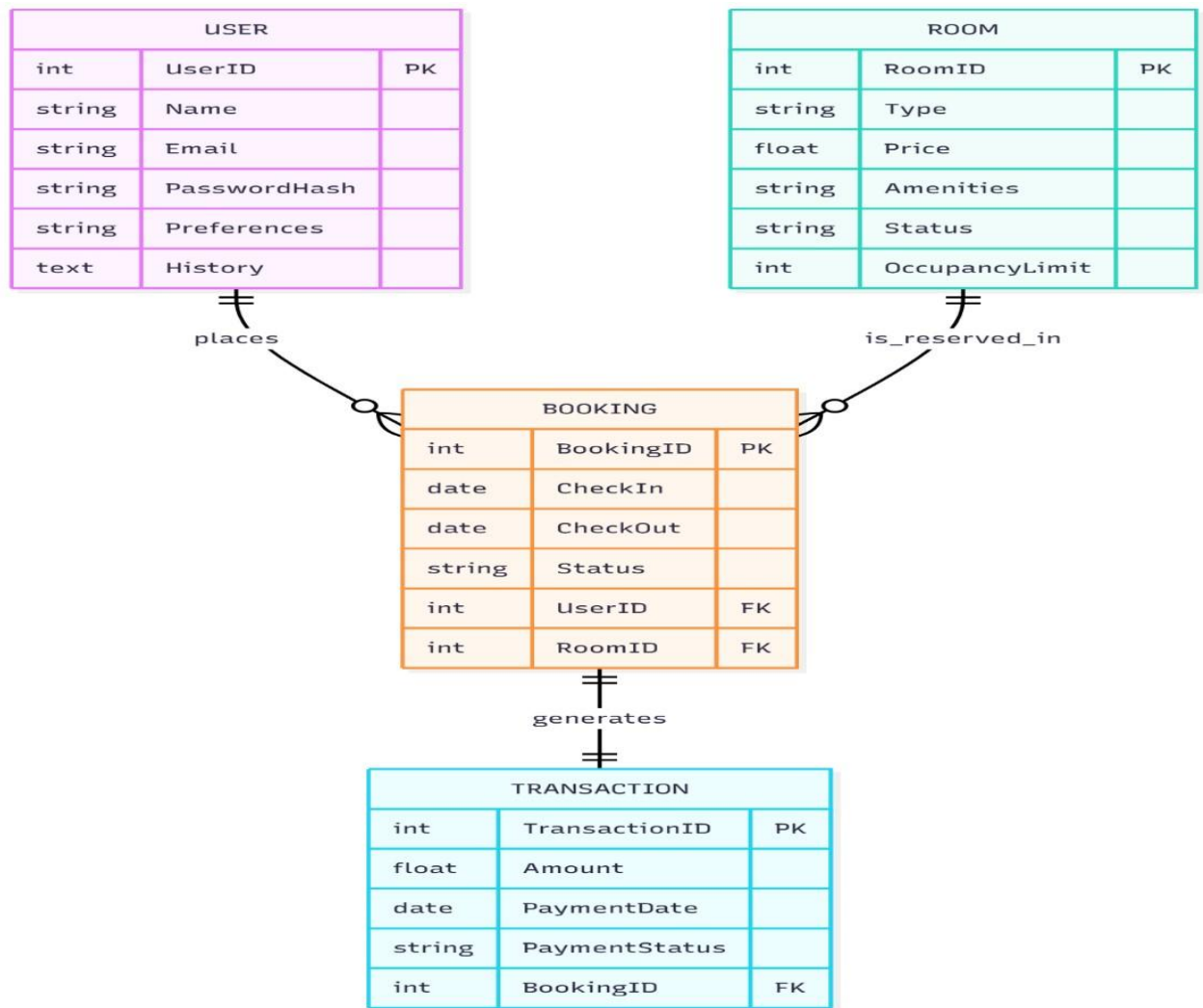
1.ER Diagram

The ER diagram illustrates the **database structure of the Hotel Management and Smart Room Booking System (SmartBook)**. It defines the key entities required to support room booking and hotel management operations.

The main entities include **User**, **Room**, **Booking**, and **Payment**. A user (guest or administrator) can create multiple bookings, while each booking is linked to one room and one payment record. Room information and booking status are maintained to ensure accurate availability and prevent double bookings.

The **AI Recommendation Engine** is not modeled as a database entity; instead, it functions as a backend logical component that utilizes data from users, rooms, and bookings to generate personalized room recommendations and upsell suggestions.

This ER design ensures **data integrity, scalability, and efficient system operation**.



Change Management Process

The Change Management Process ensures that any modifications to the requirements, design, or code of the **Hotel Management and Smart Room Booking System** are controlled, coordinated, and documented. This minimizes the risk of scope creep and ensures alignment between the student (developer) and the project supervisor.

1. Change Control Policy

All changes to the baseline requirements defined in this SRS must follow a structured procedure:

Identification: A change request can be initiated by the Developer (student) due to technical constraints or by the Supervisor (Dr. Imran Sarwar Bajwa) for requirement refinement.

Impact Analysis: Before implementation, the developer must evaluate the impact of the change on the **AI Engine**, database schema, and project timeline.

Classification:

Minor Changes: (e.g., UI color adjustments, typo fixes) Do not require formal approval but must be logged in the version history.

Major Changes: (e.g., adding a new Use Case, altering the AI logic) Require explicit approval from the Supervisor.

2. Version Control Strategy

To maintain consistency, versioning will be applied to both the documentation and the software artifacts.

Document Versioning:

The SRS document uses a two-digit format (**V 1.0**).

1.0: Initial accepted draft.

1.1, 1.2: Minor updates (clarifications, formatting).

2.0: Major updates (addition of functional modules).

All changes must be recorded in the **Revision History** table with the Date, Description, and Author.

Software Versioning:

The application code (Python/Flask) uses Semantic Versioning (**Major.Minor.Patch**).

Example: v1.0.1 (Bug fix in login), v1.1.0 (New AI feature added).

3. Approval Workflow

The workflow for approving a Major Change is as follows:

1. **Request:** The change is proposed and documented.

2. **Review:** The Supervisor reviews the proposal for feasibility and alignment with FYP objectives.
3. **Approval:** If approved, the Supervisor signs off (verbally or via email).
4. **Implementation:** The developer updates the SRS document first, then updates the code.
5. **Verification:** The change is tested to ensure no regression errors occurred in existing modules (e.g., Booking Engine).

A. Appendices

This section includes supplementary information, a glossary of terms, and technical references to support the development and maintenance of the SmartBook system.

A.1 Glossary

Administrator (Admin): Hotel staff members responsible for managing room inventory, pricing, availability, booking records, and viewing reports through the admin dashboard.

AI Engine (AI Recommendation Engine): A backend decision-support component integrated into the application layer that analyzes user behavior, booking history, and room attributes to generate personalized room recommendations and upsell suggestions. The AI logic operates in the background and is not directly exposed to end-users.

Black Box: A system design approach in which the internal processing and decision-making logic (such as AI recommendations) are hidden from the user, exposing only system inputs and outputs through the user interface.

Guest: A general public user who interacts with the system to search for available rooms, view recommendations, make bookings, and complete payments.

SRS: Software Requirements Specification; the document that defines the functional and non-functional requirements of the SmartBook system.

Upselling: A sales technique where the system encourages users to select higher-value room options or additional services based on recommendations (e.g., premium room upgrades or added amenities).

A.2 Technology Stack Details

The following technologies and tools are recommended for the development and deployment of the SmartBook system:

Frontend Technologies:

HTML5, CSS3, and JavaScript are used to implement the web-based user interface, ensuring responsiveness, interactivity, and cross-browser compatibility.

Backend Language:

PHP is used for implementing server-side business logic, authentication, booking workflows, AI orchestration, and communication with the database.

Web Server:

Apache or Nginx with PHP support.

Database:

MySQL (v8.0 or higher) or PostgreSQL for relational data storage.

AI Recommendation Logic:

The AI recommendation functionality is implemented as backend logic that processes user data, booking history, and room attributes to generate personalized recommendations. The AI operates as a logical component and does not require a separate database entity.

Database Access Layer:

PHP PDO or MySQLi is used to securely interact with the database using prepared statements.

A.3 Sample Data Structures

To assist developers in understanding the data flow for the AI Room Recommendation System, the following JSON structure serves as a reference for the **User Profile** data passed internally to the AI recommendation logic: